Lecture vocale

Contributor Guide · Guide du contributeur

English

1. How to support the project
2. How the project is organised
3. Architecture
4. Set up and test
5. How to contribute
6. Common changes
7. Publishing a version

Français

1. Comment soutenir le projet
2. Organisation du projet
3. Architecture
4. Installer et tester
5. Comment contribuer
6. Modifications courantes
7. Publier une version

Version 1.3.0 · XGEOSOFT CORPORATION · MIT License
github.com/XGEOSOFT-CORPORATION/lecture-vocale

Contributor Guide

Lecture vocale reads LibreOffice documents aloud, offline, on Windows, Linux and macOS. This guide explains how to help, how the code is organised, and how to get a change accepted.

1. How to support the project

You do not need to write code to help. Each of these makes a real difference:

- **Give the project a star** on GitHub. It encourages the team and helps other people find the extension.
- **Report problems** in the issue tracker. See how to write a useful report.
- **Test on macOS.** This is the most useful help right now: the macOS code has never been run on a real Mac.
- **Review a translation.** The interface exists in six languages, and native speakers have not reviewed them yet.
- **Add a language** to the interface, or try voices in your own language and tell us what you hear.
- **Leave a review** on the LibreOffice extensions site, and tell people who would benefit, for example people with reading difficulties.

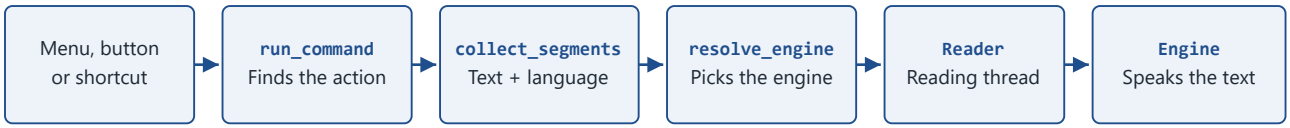
2. How the project is organised

Path	Content
`extension/`	Everything that goes into the `.oxt` file
`extension/python/lecture_vocale.py`	All the extension's code, in one file
`extension/python/i18n/*.json`	Interface translations, one file per language
`extension/Addons.xcu`	Menu and toolbar
`extension/Accelerators.xcu`	Keyboard shortcuts
`extension/WindowState/`	Toolbar name and position in each application
`extension/description.xml`	Identifier, version, minimum LibreOffice version
`extension/META-INF/manifest.xml`	List of the components LibreOffice must load
`build.py`	Checks the sources and builds the `.oxt`
`tests/`	Automated tests and checks in a real LibreOffice
`.github/workflows/`	Tests run by GitHub
`docs/`	This guide and its source. The PDF is also included in the `.oxt`, and opens from the About window

The extension has **no third-party Python dependency**. LibreOffice ships its own Python without pip, so the code may only use the standard library and the `uno` module.

3. Architecture

From a click to the sound



1. Each menu item calls a URL such as `service:com.adanalytics.LectureVocale?toggle`. LibreOffice passes the word after `?` to `LectureVocale.trigger`, which calls `run_command`.
2. `collect_segments` cuts the document into *segments*: tuples of (text, object to highlight, language).
3. `resolve_engine` takes the first engine with a voice for every language found.
4. `Reader`, a background thread, reads the segments one by one, so LibreOffice stays responsive.

Sections of lecture_vocale.py

The file is divided by comment banners, in French, in this order:

Section	Role
Configuration et journal	Settings file and log
Détection des outils	Finds programs, picks voices, recognises writing systems
Moteurs de synthèse	The `Engine` class and one subclass per engine
Outils UNO	Main thread calls, messages, interface language, menu titles
Extraction du texte à lire	Segments for Writer, Calc, Impress and Draw
Lecture en arrière-plan	`Reader`, then start, pause and jump functions
Réglages rapides, Export audio	Live settings and WAV, MP3 or OGG export
Panneau, Boîte de dialogue, À propos	`ReadingPanel`, `SettingsDialog`, `AboutDialog`
Commandes	`run_command` and the UNO entry point

Speech engines

Class	Used on	How it speaks
`SapiEngine`	Windows	PowerShell, with the modern Windows speech API first, then System.Speech
`SpdEngine`	Linux	`spd-say` (Speech Dispatcher)
`MacSayEngine`	macOS	`say`
`PiperEngine`	All	Writes a WAV file, then plays it
`EspeakEngine`	All	`espeak-ng`

In automatic mode the system's speech service comes first, then Piper, then eSpeak NG. This order is `ENGINE_KEYS`.

Rules that keep the code working

- **Automatic mode follows the language.** `pick_voice` never returns a voice of another language. A voice chosen in the list always wins: it reads everything.
- **Never fail in silence.** When no voice exists for a language, the engine records a warning, and the user sees it once.
- **Interface windows belong to the main thread.** From `Reader`, go through `run_in_main`.
- **Pause stops the speech process** and remembers the sentence. Resuming reads that sentence again.

4. Set up and test

You need Git and Python 3.8 or later. Nothing else is required to build or run the automated tests.

```
git clone https://github.com/XGEOSOFT-CORPORATION/lecture-vocale.git
cd lecture-vocale
python -m unittest discover -s tests
python build.py
```

The extension is written to `dist/LectureVocale-<version>.oxt`. To try it, install it with **Tools ▸ Extensions** and restart LibreOffice. To remove it from a test profile:

```
unopkg remove com.adanalytics.lecturevocale
```

The three levels of tests

Test	What it proves
`tests/test_voices.py`	Voice choice, languages, translations, settings. No LibreOffice needed. On Windows it also renders real audio.
`tests/libreoffice_check.py`	Runs the installed extension in a real LibreOffice: menus, shortcuts, pause, export, languages, and a full reading with every available voice.
`tests/speech_dispatcher_check.py`	Records what Speech Dispatcher really outputs, and checks that voice, pitch and speed change the sound.

The last two run in one command on Linux, with Docker. The first run downloads about 1 GB:

```
docker build -t lvlo-linux-test tests/linux
docker run --rm -v "$PWD:/repo:ro" lvlo-linux-test bash /repo/tests/linux/run.sh
```

To run `libreoffice_check.py` elsewhere, follow the instructions at the top of the file. Use LibreOffice's own Python on Windows and macOS. On a Mac, `python3 tests/macos_check.py` checks the real voices.

GitHub runs the automated tests on Windows, Linux and macOS, the Linux check and the macOS check, for every push and pull request. The result appears on your pull request, and each run offers the `.oxt` to download.

Tip. These tests found bugs that the automated tests could not see, such as shortcuts that were never registered. Run them before any change to menus, shortcuts or dialogs.

5. How to contribute

Report a problem

Open an issue and include:

- your operating system and LibreOffice version;
- the extension version and the speech engine, both shown in the extension's windows;
- what you did, what you expected, and what happened;
- the end of the log file.

System	Log and settings folder
Windows	`%APPDATA%\lecture-vocale-libreoffice`
Linux, macOS	`~/.config/lecture-vocale-libreoffice`

The log shows the languages and voice used for each reading. Check that it holds no private text before you share it.

Propose a change

1. For a large change, open an issue first to discuss it.
2. Fork the repository and create a branch, for example `fix/pause-on-linux`.
3. Make one focused change, and add or update a test.
4. Run the tests and `python build.py`.
5. Open a pull request: say what changes, why, and on which system you tested.

Code rules

- **Python 3.8 syntax.** LibreOffice 24.2 ships Python 3.8; `build.py` rejects newer syntax. The code must also run on recent versions, tested up to Python 3.13.
- **Standard library only.**
- **Every visible text goes through `_()`,** in French, as a whole sentence. Do not build sentences from fragments: they cannot be translated.
- **Never use `_` as a throwaway variable.** It would hide the translation function. A test checks this.
- **Match the surrounding code:** comments are in French, and they explain why, not what.

Contributions are published under the project's MIT license.

6. Common changes

Add an interface language

1. Copy `extension/python/i18n/en.json` to `<code>.json`. Translate the values, never the keys, and keep every `%s` and `%d`.
2. Add the language to `UI_LANGUAGES` in `lecture_vocale.py`.
3. Add a `xml:lang` value to each title in `Addons.xcu`.
4. Add its code to `test_menus_are_available_in_every_language`, then run the tests. They check that nothing is missing.

Add a speech engine

1. Subclass `Engine`, with a `key` and a `label`.
2. Implement `speak_command`, or `synth` with `file_based = True`. Add `supports` if the engine can list its voices.
3. Add it to `ENGINES`, `ENGINE_KEYS` and `_ENGINE_LABELS`, and translate the label.
4. Implement `voices`, so that its voices appear in the list, and add tests for its commands.

Add a menu command

1. Add the item to `Addons.xcu`, in every language.
2. Handle the command in `run_command`, and add its title to `MENU_TITLES`.
3. For a shortcut, use `Accelerators.xcu`. Check that no application already uses the key: `libreoffice_check.py` tests this.

7. Publishing a version

1. Change the version in `description.xml` and in `VERSION`. A test checks that they match.
2. Update the changelog in `README.md`, in both languages.
3. If this guide changed, run `python docs/build_guide.py`.
4. Run all three levels of tests, on LibreOffice 24.2 and on the latest version.
5. Run `python build.py`, and publish the `.oxt` on GitHub and on extensions.libreoffice.org.

Never change the identifier `com.adanalytics.lecturevocale`. LibreOffice uses it to replace the previous version. With a new identifier, the update would install as a second extension.

Guide du contributeur

Lecture vocale lit à voix haute les documents LibreOffice, hors ligne, sous Windows, Linux et macOS. Ce guide explique comment aider, comment le code est organisé et comment faire accepter une modification.

1. Comment soutenir le projet

Pas besoin d'écrire du code pour aider. Chacune de ces actions compte vraiment :

- **Donnez une étoile au projet** sur GitHub. Cela encourage l'équipe et aide d'autres personnes à trouver l'extension.
- **Signalez les problèmes** dans les tickets. Voyez comment rédiger un signalement utile.
- **Testez sous macOS.** C'est l'aide la plus utile aujourd'hui : le code macOS n'a jamais tourné sur un vrai Mac.
- **Relisez une traduction.** L'interface existe en six langues, et aucun locuteur natif ne les a encore relues.
- **Ajoutez une langue** à l'interface, ou essayez les voix de votre langue et dites-nous ce que vous entendez.
- **Laissez un avis** sur le site des extensions LibreOffice, et parlez-en aux personnes à qui l'extension peut servir, par exemple celles qui ont des difficultés de lecture.

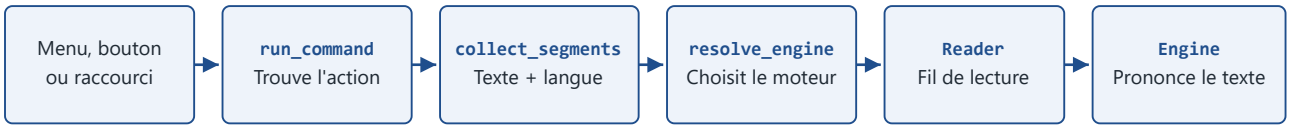
2. Organisation du projet

Chemin	Contenu
`extension/`	Tout ce qui entre dans le fichier `.oxt`
`extension/python/lecture_vocale.py`	Tout le code de l'extension, en un seul fichier
`extension/python/i18n/*.json`	Traductions de l'interface, un fichier par langue
`extension/Addons.xcu`	Menu et barre d'outils
`extension/Accelerators.xcu`	Raccourcis clavier
`extension/WindowState/`	Nom et position de la barre d'outils dans chaque application
`extension/description.xml`	Identifiant, version, version minimale de LibreOffice
`extension/META-INF/manifest.xml`	Liste des composants que LibreOffice doit charger
`build.py`	Vérifie les sources et construit le `.oxt`
`tests/`	Tests automatiques et vérifications dans un vrai LibreOffice
`.github/workflows/`	Tests lancés par GitHub
`docs/`	Ce guide et sa source. Le PDF est aussi inclus dans le `.oxt`, et s'ouvre depuis la fenêtre À propos

L'extension n'a **aucune dépendance Python externe**. LibreOffice embarque son propre Python, sans pip : le code ne peut utiliser que la bibliothèque standard et le module `uno`.

3. Architecture

Du clic au son



1. Chaque entrée de menu appelle une adresse comme `service:com.adanalytics.LectureVocale?toggle`. LibreOffice transmet le mot après `?` à `LectureVocale.trigger`, qui appelle `run_command`.
2. `collect_segments` découpe le document en *segments* : des triplets (texte, objet à surligner, langue).
3. `resolve_engine` prend le premier moteur qui a une voix pour toutes les langues trouvées.
4. `Reader`, un fil d'exécution en arrière-plan, lit les segments un à un : LibreOffice reste réactif.

Sections de lecture_vocale.py

Le fichier est découpé par des bandeaux de commentaires, dans cet ordre :

Section	Rôle
Configuration et journal	Fichier de réglages et journal
Détection des outils	Trouve les programmes, choisit les voix, reconnaît les écritures
Moteurs de synthèse	La classe `Engine` et une sous-classe par moteur
Outils UNO	Appels au fil principal, messages, langue de l'interface, titres des menus
Extraction du texte à lire	Segments pour Writer, Calc, Impress et Draw
Lecture en arrière-plan	`Reader`, puis les fonctions de démarrage, pause et saut
Réglages rapides, Export audio	Réglages en direct et export WAV, MP3 ou OGG
Panneau, Boîte de dialogue, À propos	`ReadingPanel`, `SettingsDialog`, `AboutDialog`
Commandes	`run_command` et le point d'entrée UNO

Moteurs vocaux

Classe	Utilisée sous	Comment elle parle
`SapiEngine`	Windows	PowerShell, avec d'abord l'API vocale moderne de Windows, puis System.Speech
`SpdEngine`	Linux	`spd-say` (Speech Dispatcher)
`MacSayEngine`	macOS	`say`

`PiperEngine`	Tous	Écrit un fichier WAV, puis le joue
`EspeakEngine`	Tous	`espeak-ng`

En mode automatique, le service vocal du système passe en premier, puis Piper, puis eSpeak NG. Cet ordre est celui de `ENGINE_KEYS`.

Les règles qui font tenir le code

- **Le mode automatique suit la langue.** `pick_voice` ne renvoie jamais une voix d'une autre langue. Une voix choisie dans la liste passe toujours en premier : elle lit tout.
- **Jamais d'échec silencieux.** Quand aucune voix n'existe pour une langue, le moteur enregistre un avertissement, que l'utilisateur voit une fois.
- **Les fenêtres appartiennent au fil principal.** Depuis `Reader`, passez par `run_in_main`.
- **La pause arrête le processus vocal** et mémorise la phrase. La reprise relit cette phrase.

4. Installer et tester

Il vous faut Git et Python 3.8 ou plus récent. Rien d'autre n'est nécessaire pour construire l'extension et lancer les tests automatiques.

```
git clone https://github.com/XGEOSOFT-CORPORATION/lecture-vocale.git
cd lecture-vocale
python -m unittest discover -s tests
python build.py
```

L'extension est écrite dans `dist/LectureVocale-<version>.oxt`. Pour l'essayer, installez-la avec **Outils** ▸ **Extensions** et redémarrez LibreOffice. Pour la retirer d'un profil de test :

```
unopkg remove com.adanalytics.lecturevocale
```

Les trois niveaux de tests

Test	Ce qu'il prouve
`tests/test_voices.py`	Choix des voix, langues, traductions, réglages. Sans LibreOffice. Sous Windows, il produit aussi de l'audio réel.
`tests/libreoffice_check.py`	Fait tourner l'extension installée dans un vrai LibreOffice : menus, raccourcis, pause, export, langues, et une lecture complète avec chaque voix disponible.
`tests/speech_dispatcher_check.py`	Enregistre ce que Speech Dispatcher produit réellement, et vérifie que la voix, la tonalité et le débit changent le son.

Les deux derniers se lancent en une commande sous Linux, avec Docker. Le premier lancement télécharge environ 1 Go :

```
docker build -t lvlo-linux-test tests/linux
docker run --rm -v "$PWD:/repo:ro" lvlo-linux-test bash /repo/tests/linux/run.sh
```

Pour lancer `libreoffice_check.py` ailleurs, suivez les instructions en tête du fichier. Sous Windows et macOS, utilisez le Python de LibreOffice. Sur un Mac, `python3 tests/macos_check.py` vérifie les vraies voix.

GitHub lance les tests automatiques sous Windows, Linux et macOS, la vérification Linux et la vérification macOS, à chaque envoi et à chaque *pull request*. Le résultat s'affiche sur votre *pull request*, et chaque exécution propose le `.oxt` en téléchargement.

> **Conseil.** Ces tests ont trouvé des bugs invisibles pour les tests automatiques, comme des raccourcis jamais enregistrés. Lancez-les avant toute modification des menus, des raccourcis ou des fenêtres.

5. Comment contribuer

Signaler un problème

Ouvrez un ticket en indiquant :

- votre système et votre version de LibreOffice ;
- la version de l'extension et le moteur vocal, visibles dans ses fenêtres ;
- ce que vous avez fait, ce que vous attendiez et ce qui s'est passé ;
- la fin du fichier journal.

Système	Dossier du journal et des réglages
Windows	`%APPDATA%\lecture-vocale-libreoffice`
Linux, macOS	`~/.config/lecture-vocale-libreoffice`

Le journal indique les langues et la voix de chaque lecture. Vérifiez qu'il ne contient aucun texte privé avant de le partager.

Proposer une modification

1. Pour une grosse modification, ouvrez d'abord un ticket pour en discuter.
2. Créez une copie (*fork*) du dépôt et une branche, par exemple `fix/pause-linux`.
3. Faites une seule modification ciblée, et ajoutez ou adaptez un test.
4. Lancez les tests et `python build.py`.
5. Ouvrez une *pull request* : dites ce qui change, pourquoi, et sur quel système vous avez testé.

Règles de code

- **Syntaxe Python 3.8.** LibreOffice 24.2 embarque Python 3.8 ; `build.py` refuse une syntaxe plus récente. Le code doit aussi tourner sur les versions récentes, testé jusqu'à Python 3.13.
- **Bibliothèque standard uniquement.**
- **Tout texte visible passe par `_()`,** en français, sous forme de phrase entière. Ne construisez pas de phrases par morceaux : elles seraient intraduisibles.
- **N'utilisez jamais `_` comme variable jetable.** Cela masquerait la fonction de traduction. Un test le vérifie.

- **Suivez le code existant :** les commentaires sont en français et expliquent pourquoi, pas quoi.

Les contributions sont publiées sous la licence MIT du projet.

6. Modifications courantes

Ajouter une langue d'interface

1. Copiez `extension/python/i18n/en.json` vers `<code>.json`. Traduisez les valeurs, jamais les clés, et gardez chaque `%s` et `%d`.
2. Ajoutez la langue à `UI_LANGUAGES` dans `lecture_vocale.py`.
3. Ajoutez une valeur `xml:lang` à chaque titre de `Addons.xcu`.
4. Ajoutez son code à `test_menus_are_available_in_every_language`, puis lancez les tests : ils vérifient que rien ne manque.

Ajouter un moteur vocal

1. Créez une sous-classe de `Engine`, avec une `key` et un `label`.
2. Implémentez `speak_command`, ou `synth` avec `file_based = True`. Ajoutez `supports` si le moteur sait lister ses voix.
3. Ajoutez-le à `ENGINES`, `ENGINE_KEYS` et `_ENGINE_LABELS`, et traduisez son libellé.
4. Implémentez `voices`, pour que ses voix apparaissent dans la liste, et ajoutez des tests pour ses commandes.

Ajouter une commande de menu

1. Ajoutez l'entrée à `Addons.xcu`, dans toutes les langues.
2. Traitez la commande dans `run_command` et ajoutez son titre à `MENU_TITLES`.
3. Pour un raccourci, utilisez `Accelerators.xcu`. Vérifiez qu'aucune application n'utilise déjà la touche : `libreoffice_check.py` le teste.

7. Publier une version

1. Changez la version dans `description.xml` et dans `VERSION`. Un test vérifie qu'elles concordent.
2. Mettez à jour l'historique dans `README.md`, dans les deux langues.
3. Si ce guide a changé, lancez `python docs/build_guide.py`.
4. Lancez les trois niveaux de tests, sur LibreOffice 24.2 et sur la dernière version.
5. Lancez `python build.py`, puis publiez le `.oxt` sur GitHub et sur extensions.libreoffice.org.

> **Ne changez jamais l'identifiant** `com.adanalytics.lecturevocale`. LibreOffice s'en sert pour remplacer la version précédente. Avec un nouvel identifiant, la mise à jour s'installerait comme une seconde extension.